Kotlin

- 基础 ㉘
- OOP ㉙
- 集合 ㊷
- 函数式编程
- 多线程 ③
- 设计模式 ⑨
- 协程
  - 概述
    - 概念
      - 协程（coroutines）允许执行耗时任务的同时，不阻塞线程。这些任务在称为挂起函数（suspend）内部执行。suspend函数能够在执行中途暂停，然后从函数被暂停的地方恢复执行。
    - 原理
      - 底层通过回调实现，对开发者隐藏。开发者可以像开发同步代码一样（替代之前回调的异步实现），使代码可读性更好。
  - 创建协程
    - 通过某个作用域对象调用launch函数创建并启动一个协程

      -
        ```
        GlobalScope.launch{
            // 内部可以调用suspend函数
        }
        ```
      - 实践中建议使用自定义作用域

        -
          ```
          // 创建自定义协程，指定名称与调度器
          private val scope =
          CoroutineScope(CoroutineName("My Coroutine", Dispatchers.Undefined));
          ```
      - Job
        - 协程launch函数返回Job实例，Job代表一个协程任务。可以通过Job操作协程的启动、取消等。

          -
            ```
            val job = scope.launch{}
            job.cancel() // 取消
            job.join() // 等待执行完成
            ```
          - 使用joinAll()批量等待Job执行完成

            -
              ```
              val jobs = mutableListOf<Job>()
              repeat(10) {
                  scope.launch{}.also{jobs.add(it)}
              }
              jobs.joinAll()
              ```
    - 使用async创建并启动一个协程
      - 返回Deferred<T>实例（Job的子类）提供了替代join的await()方法，返回T类型

        -
          ```
          val deferredJob = scope.async{
              Book("title") // 最后一行返回
          }
          val book = deferredJob.await()
          ```

        -
          ```
          val deferredJobs = mutableListOf(Deferred<Book>)
          repeat(10) {
              scope.async {
                  Book("title")
              }
          }
          // 使用awaitAll等待所有协程执行完成并获取结果到集合
          val books: List<Book> = deferredJobs.awaitAll();
          ```
  - 结构化并发原则
    - 所有协程都应该在一个具有明确生命周期的作用域内启动
      - 这意味着如果不需要某些协程，应该有方法取消该作用域。而GlobalScope作用域是无法取消的，故实践中不推荐使用

        -
          ```
          示例：
          private val scope = CoroutineScope();
          // 在作用域内启动一个协程
          scope.launch {}
          // 取消作用域内所有协程
          scope.cancel()
          ```
    - 协程具有层级关系，父协程取消后，所有子协程也会取消；父协程在所有子协程结束前不会结束

      -
        ```
        scope.launch {
            // 启动第一个子协程
            scope.launch {}
            // launch内部默认this，可省略scope.launch {}
        }
        ```
    - 如果子协程抛出异常，则会抛出给父协程，如果父协程没有处理该异常，程序会崩溃
  - 调度器
    - 决定协程在哪个线程执行
    - Dispatchers.Default
      - 默认调度器，使用后台线程池执行线程数量 = CPU核心数
    - Dispatchers.Unconfined
      - 非受限调度器，初始在调用线程执行，恢复时由挂起函数决定线程
    - Dispatchers.IO
      - 内部使用cached线程池，最多支持64个线程
    - Dispatchers.Main
      - 确保在主线程执行，用于android UI更新
    - 自定义调度器
      - 默认的调度器创建的线程都是守护线程，当所有非守护线程结束时，守护线程会自动停止
        - 可以将线程池转换为协程调度器，默认情况下线程池的线程不是守护线程，例如：
          ```
          val dispatcher =
          Executors.newSingleThreadExecutor().asCoroutineDispatcher();
          val scope = CoroutineScope(dispatcher);
          ```

          -
            ```
            fun main() {
                scope.launch {
                    while(true) {
                        println(1)
                        Thread.sleep(1000)
                    }
                }
            }
            ```
            - 如果scope是默认的守护线程，则1只打印一次或不打印
              如果是自定义的非守护线程，会无限打印下去
  - suspend函数
    - 只能从其他挂起函数，或协程调用
      - `suspend fun getUrlData() : string {}`
    - suspend函数内部不能执行耗时操作，如果所在的作用域调度器是Dispatchers.Unconfined，则可能导致调用方阻塞
      - 如果suspend函数中有耗时操作，建议通过withContext()切换上下文（这会导致执行挂起函数后切换线程）
    - 挂起与恢复
      - 协程使得代码看起来像是顺序执行的，而实际上携程内每个挂起函数执行时会挂起，执行结束会从suspend函数后恢复（通过回调）
  - delay函数
    - delay是coroutine提供的非阻塞式的等待函数

      -
        ```
        val dispatcher =
        Executors.newSingleThreadExecutor().asCoroutineDispatcher();
        val scope = CoroutineScope(dispatcher);

        fun main(args: Array<String>) {
            scope.launch {
                println("AAA")
                Thread.sleep(1000)
                // delay(1000)
                println("BBB")
            }
            scope.launch {
                println("CCC")
            }
        }
        ```
        - 单线程调度器的协程，必须等一个执行完，另一个才能得到执行，打印结果：
          AAA
          BBB
          CCC
        - 使用delay(1000)，会临时释放线程，打印结果
          AAA
          CCC
          BBB
  - 处理异常
    - 通过try-catch块无法捕获launch异常，因为launch只代表协程启动成功

      -
        ```
        suspend fun method() {
            delay(1000)
            error("") // 内部throw了一个异常
        }
        // 捕获失败，协程崩溃
        try {
            method()
        } catch (e : Exception) {
            println("Exception Caught")
        }
        ```
      - 但是：不管协程的异常是捕获了（包括内部/外部捕获），还是没捕获导致协程崩溃。都不会影响其他协程/主线程导致其崩溃
    - 正确捕获
      - 1、内部捕获
        - 使用try-catch捕获

          -
            ```
            scope.launch {
                    try {
                        method()
                    } catch (e: Exception) {
                        println("Exception Caught")
                    }
                }
            ```
        - 使用runCatching

          -
            ```
            scope.launch {
                    runCatching {
                        method()
                    }.onSuccess {  }.onFailure {}
                }
            ```
      - 2、外部捕获
        - 如果协程内部不捕获异常，则会向上抛出给父协程，可以配置ExceptionHandler处理异常，则程序不会崩溃，但是作用域中的所有子协程被取消

          -
            ```
            val exceptionHandler =
            CoroutineExceptionHandler { _, _ ->
                println("Exception Caught")
            }
            // 配置协程处理器
            val scope = CoroutineScope(exceptionHandler);

            fun main() {
                scope.launch {
                    method()
                }
                // 处理异常后，子协程取消，methods2中不会打印
                scope.launch {
                    method2()
                }
            }

            suspend fun method2() {
                delay(5000)
                println("method2 invoked")
            }

            suspend fun method() {
                delay(3000)
                error("123")
            }
            ```
        - launch是直接抛出，而async是封装到Deferred抛出的，无论那种方法抛出，都会导致该scope下所有子协程取消。但是async抛出的异常在【解包】出异常前，异常无需被捕获处理

          -
            ```
            // async没有解包，无需处理异常
            fun main() {
                val job = scope.async {
                    method()
                }
            }

            suspend fun method() {
                delay(3000)
                error("123")
            }
            ```
          - 执行async.await时，解包

            -
              ```
              fun main() {
                  val deferred = scope.async {
                      method()
                  }
                  scope.launch {
                      // await时解包，launch向上抛出
                      deferred.await()
                  }
              }

              suspend fun method() {
                  delay(3000)
                  error("123")
              }
              ```
    - SupervisorJob
      - 其子协程抛出异常并处理后，SupervisorJob的其他子协程不会停止

        -
          ```
          val parentJob = SupervisorJob()
          val scope = CoroutineScope(parentJob + exceptionHandler)
          // 第一个协程抛异常后，第二个不会停止
          scope.launch { error("") }
          scope.launch { xxx }
          ```
  - 协程取消
    - scope.cancel
      - 取消scope下所有job，示例略
    - Job.cancel
      - 试图"取消"协程，其实现是将协程的isActive置为false，协程的代码中要包含对isActive的检查，并退出协程
        - // 没有处理isActive的代码，调用Job.cancel无效
        - 如何退出协程?
          判断isActive是false时，抛出CancellationException，逐层上报给scope，该类型是一种特殊的Exception，即使scope没有实现CoroutineExceptionHandler，也不会使程序崩溃，scope收到该Exception后，会将该协程退出

          -
            ```
            suspend fun timer() {
                while (true) {
                    try {
                        // suspend函数要实现协程退出的代码
                        if (!coroutineContext.isAcitve) throw CancellationException()
                        // 上面的if判断，可以直接替换为如下封装函数
                        ensureActive()
                        // 某些函数中，封装了上述逻辑，例如delay()
                        // 如果调用了delay()，可以省略isActive判断
                        delay(1000)

                        // 执行其他代码
                    } catch (e : CancellationException) {
                        // suspend中的try-catch块
                        // 一定要把CancellationException重新抛出，否则协程无法取消
                    } catch (e : Exception) {}
                }
            }
            ```
      - Job.cancel仅试图取消当前Job、以及当前Job下的子Job
        - 只有launch（注：不带scope的）才算作子Job

          -
            ```
            val job : Job = scope.launch {
                // 不属于子Job，不cancel
                scope.launch {
                    var time = 0
                    while (true) {
                        delay(1000)
                        println("inner1 $time")
                        time++
                    }
                }

                // 子Job，cancel
                launch {
                    var time = 100
                    while (true) {
                        ensureActive()
                        delay(1000)
                        println("inner2 $time")
                        time++
                    }
                }

                // Job本身，cancel
                var time = 200
                while (true) {
                    ensureActive()
                    delay(1000)
                    println("inner2 $time")
                    time++
                }
            }

            // 其他Job，不cancel
            scope.launch {
                var time = 300
                while (true) {
                    delay(1000)
                    println("inner3 $time")
                    time++
                    if (time == 305) {
                        println("cancel job")
                        job.cancel()
                    }
                }
            }
            ```

            -
              ```
              // 打印结果
              inner1 0
              inner2 100
              inner2 200
              inner3 300
              inner1 1
              inner3 301
              inner2 201
              inner2 101
              inner2 202
              inner1 2
              inner3 302
              inner2 102
              inner1 3
              inner2 103
              inner2 203
              inner3 303
              inner3 304
              inner2 104
              inner2 204
              inner1 4
              cancel job
              inner1 5
              inner3 305
              inner3 306
              inner1 6
              inner3 307
              inner1 7
              inner1 8
              inner3 308
              inner3 309
              inner1 9
              inner3 310
              ```
              - 100开头、200开头的数字，后面不打印了
    - 与异常处理的区别?
      - 本节讨论下当某个Job中发生异常、或者cancel时，对该Job、该Scope、该scope下其他job，其他scope的影响
        - 本节所有例子中：scope和scope2有配置ExceptionHandler，scope3没有配置ExceptionHandler
      - 异常的影响范围是整个scope的，与scope.cancel()类似。
        - 异常发生时
          - 1、如果被内部捕获，则无影响

            -
              ```
              // 举例
                  scope.launch {
                      scope2.launch {
                          delay(3000)
                          try {
                              error("e")
                          } catch (e: Exception) {
                              println("Exception caught inner scope2")
                          }
                          println("scope2 continue")
                      }
                      var time = 100
                      while (true) {
                          delay(1000)
                          println("inner2 $time")
                          time++
                      }
                  }
              ```

              -
                ```
                // 打印结果
                inner2 100
                inner2 101
                Exception caught inner scope2
                scope2 continue
                inner2 102
                inner2 103
                inner2 104
                inner2 105
                ```
                - scope2内部处理完异常后，scope2不停止，scope1也不受影响
          - 2、如果外部捕获，则该scope停止、该scope所有job停止，但是不影响其他scope

            -
              ```
                  scope.launch {
                      scope.launch {
                          var time = 0
                          while (true) {
                              delay(1000)
                              println("inner1 $time")
                              time++
                          }
                      }
                      var time = 100
                      while (true) {
                          delay(1000)
                          println("inner2 $time")
                          time++
                      }
                  }

                  scope.launch {
                      var time = 200
                      while (true) {
                          delay(1000)
                          println("inner3 $time")
                          time++
                          if (time == 205) error("exception")
                      }
                  }
              ```

              -
                ```
                // 打印结果
                inner3 200
                inner2 100
                inner1 0
                inner1 1
                inner2 101
                inner3 201
                inner3 202
                inner1 2
                inner2 102
                inner2 103
                inner3 203
                inner1 3
                inner2 104
                inner1 4
                inner3 204
                Exception Caught
                ```
                - 抛出异常，scope停止

            -
              ```
              scope.launch {
                  launch {
                      var time = 0
                      while (true) {
                          delay(1000)
                          println("inner1 $time")
                          time++
                      }
                  }
              }

                  scope.launch {
                      var time = 200
                      while (true) {
                          delay(1000)
                          println("inner3 $time")
                          time++
                          if (time == 205) error("exception")
                      }
                  }
              ```

              -
                ```
                // 打印结果
                inner1 0
                inner3 200
                inner3 201
                inner1 1
                inner1 2
                inner3 202
                inner3 203
                inner1 3
                inner1 4
                inner3 204
                Exception Caught
                ```
                - 注意本例中内部使用launch而不是scope.launch创建子job，但是结果是一样的，launch创建的也被停止，这与cancel是有区别的
            - 不影响其他scope，不管其他scope在什么位置

              -
                ```
                    scope.launch {
                        scope2.launch {
                            delay(3000)
                            error("e")
                        }
                        var time = 100
                        while (true) {
                            delay(1000)
                            println("inner2 $time")
                            time++
                        }
                    }
                ```

                -
                  ```
                  // 打印结果
                  inner2 100
                  inner2 101
                  Exception Caught
                  inner2 102
                  inner2 103
                  inner2 104
                  inner2 105
                  ```
                  - scope2处理完异常后，scope2停止，scope2外部的scope继续执行

              -
                ```
                scope.launch {
                    scope2.launch {
                        var time = 0
                        while (true) {
                            delay(1000)
                            println("inner1 $time")
                            time++
                        }
                    }
                }

                    scope.launch {
                        var time = 200
                        while (true) {
                            delay(1000)
                            println("inner3 $time")
                            time++
                            if (time == 205) error("exception")
                        }
                    }
                ```

                -
                  ```
                  // 打印结果
                  inner1 0
                  inner3 200
                  inner3 201
                  inner1 1
                  inner3 202
                  inner1 2
                  inner3 203
                  inner1 3
                  inner3 204
                  inner1 4
                  Exception Caught
                  inner1 5
                  inner1 6
                  inner1 7
                  inner1 8
                  inner1 9
                  ```
                  - scope处理完异常后，scope停止，scope内部包含的scope2继续执行
          - 3、如果没有捕获，异常抛出，该scope崩溃，不影响其他scope

            -
              ```
                  scope.launch {
                      scope3.launch {
                          delay(3000)
                          error("e")
                      }
                      var time = 100
                      while (true) {
                          delay(1000)
                          println("inner2 $time")
                          time++
                      }
                  }
              ```

              -
                ```
                // 打印结果
                inner2 100
                inner2 101
                inner2 102
                Exception in thread "pool-2-thread-1" java.lang.IllegalStateException: e
                inner2 103
                inner2 104
                inner2 105
                ```
                - scope3没有异常处理器，内部也没有捕获异常，scope3停止。但是scope3崩溃，不会导致其他scope崩溃
      - 而Job的影响范围是当前Job及其子Job
    - coroutineScope
      - 实践中Job.cancel的使用率远高于Scope.cancel，因为后者影响范围更大
        - 如果我们需要实现一个库函数，其是挂起函数，且内部创建了新的Job。
          1、如果库函数的实现直接使用库内部的scope创建Job，那么当外部调用者期望停止任务时，不能控制库函数内部的Job停止
          2、但是我们又无法在库函数内部获取到调用者的scope，该怎么实现呢?
      - 可以通过coroutineScope实现

        -
          ```
          suspend fun repositoryMethod() {
              coroutineScope {
              // superviserScope { // FOR superviserJob
                  // 可以随着调用者的Job的cancel而取消
                  launch {
                      // XX实现代码
                  }
              }
          }
          ```
  - 切换上下文
    - 方法一：withContext

      -
        ```
        suspend fun method() {
            withContext(Dispatchers.IO) {
                // 切换调度器
            }
            // 后面的代码在新的调度器线程工作
            // withContext不能切换CoroutineExceptionHandler
        }
        ```
    - 方法二：launch入参

      -
        ```
        // 配置异常处理
        scope.launch(exceptionHandler) {
                method()
            }
        // 配置异常处理&调度器
        scope.launch(Dispatchers.Default + exceptionHandler) {
                method()
            }
        ```
  - 处理回调函数
    - 某些回调函数不可修改（例如位于库中），为了使其也能按照【顺序】的方式编写，需要将【回调函数】转化为【挂起函数】

      -
        ```
        // 将回调函数封装为挂起函数
        suspend fun loadBook(): Book {
            // suspendCoroutine执行回调函数后会释放线程
            // 在回调中执行resumeWith重新执行
            return suspendCoroutine<Book> { continuation: Continuation<Book> ->
                // 调用回调函数
                loadBook { book: Book
                    continuation.resumeWith(Result.success(book))
                }
            }
        }
        ```
  - 同步
    - 挂起函数不能使用synchronized进行同步，原因是synchronized会占有线程一段时间，而suspend是会释放线程，一段时间后重新执行的
    - 为保障协程内部的临界区同步，通过Mutex实现Mutex保障临界区代码在同一个协程内部，同一时间仅能有一个Job执行

      -
        ```
        class Counter {
            var time = 0

            suspend fun inc() {
                val mutex = Mutex();
                mutex.withLock {
                    delay(1)
                    time++
                }
            }
        }
        ```

        -
          ```
          class Counter {
              var time = 0

              suspend fun inc() {
                  val mutex = Mutex();
                  mutex.lock()
                  try {
                      delay(1)
                      time++
                  } finally {
                      mutex.unlock()
                  }
              }
          }
          ```
- 流 …
- 语法糖一 ㊲
- 语法糖二 ⑰
- 泛型 ㉝
- 内联函数 ⑲
- 数据类 ⑬
- 委托 ⑭